

Below is a clean, interview-focused Git + GitHub guide. I am also fixing the weak parts of your notes, because in an interview vague answers get exposed fast.

1) Git vs GitHub

What it is

Git is a version control system on your machine.

It tracks changes in files, lets you go back to old versions, and helps multiple people work safely.

GitHub is a cloud platform that stores Git repositories online.

It is used for sharing code, collaboration, review, issues, pull requests, and deployment.

How to say it in interview

“Git is the version control tool I use locally to track code history. GitHub is the online platform where I push my Git repository for collaboration, backup, and deployment.”

Example

You build a project:

- Git keeps local history like: v1, v2, v3
- GitHub stores that history online and lets teammates work on it

Common trap

Do not say:

- “Git and GitHub are the same”
- “GitHub is the tool that tracks versions locally”

That is wrong.

2) Why version control is needed

Problem it solves

Without version control, people make messy file copies:

- report_final.doc
- report_final2.doc
- report_really_final.doc

That is chaos.

Version control gives:

- history of changes
- rollback to old versions
- teamwork without overwriting each other
- accountability: who changed what and when

How to say it in interview

“Version control solves file chaos. It keeps a clean history of changes, supports collaboration, and lets us restore previous versions if something breaks.”

Example

Suppose your code works yesterday, but today it crashes after changes. With Git, you can compare versions and go back to the last working commit.

3) Repository

Meaning

A **repository** is a project folder tracked by Git.

It contains:

- your code
- commit history
- branches
- Git metadata in .git

How to say it

“A repository is a project directory where Git stores code history and version information.”

Example

my-website/ becomes a Git repository after git init.

4) Basic local Git workflow

This is the core flow you must know:

Edit files → **stage** → **commit** → **push**

4.1 git init

Creates a new Git repository in the current folder.

Example

```
git init
```

Interview line

“git init turns a normal folder into a Git repository.”

4.2 git status

Shows current state of files:

- untracked
- modified
- staged
- ready to commit

Example

```
git status
```

Interview line

“I use git status to check what changed and what is staged before committing.”

4.3 git add

Moves changes to the staging area.

Examples

```
git add index.html  
git add .  
git add -A
```

Difference

- git add index.html → stage one file
- git add . → stage changes in current folder
- git add -A → stage all changes including deletions

Interview line

“git add prepares changes for the next commit by moving them to staging.”

4.4 git commit

Saves a snapshot of staged changes.

Example

```
git commit -m "Add navbar and footer"
```

Interview line

“A commit is a saved snapshot of staged changes with a meaningful message.”

Good commit message style

Bad:

```
git commit -m "done"
```

Good:

```
git commit -m "Fix login validation bug"
```

4.5 git log

Shows commit history.

Example

```
git log  
git log --oneline
```

Interview line

“git log helps me trace project history and identify when changes were introduced.”

5) First-time Git setup

If Git asks “Who are you?”, it means it needs your identity for commit tracking.

Commands

```
git config --global user.name "Your Name"  
git config --global user.email "your@email.com"
```

Check config

```
git config --global --list
```

Interview line

“Git stores author information in commits, so I set my username and email using git config before my first commit.”

Important correction

Use --global if you want the settings for your whole machine. Without it, settings apply only to the current repository.

6) Remote repository and GitHub connection

```
git remote add origin <url>
```

Connects your local repo to GitHub.

Example

```
git remote add origin https://github.com/username/repo.git
```

Check remote

```
git remote -v
```

Interview line

“I connect my local Git repository to GitHub using a remote named origin.”

What origin means

Just a common alias for the remote repository URL.

7) Push and Pull

```
git push
```

Sends local commits to GitHub.

Example

git push origin main

git pull

Brings remote changes from GitHub to your local machine.

Example

git pull origin main

How to say it

“push uploads my local commits to the remote repository, and pull downloads and merges remote changes into my local branch.”

Simple analogy

- Push = upload
 - Pull = download + merge
-

8) Branching

This is one of the most important interview topics.

What is a branch?

A branch is an independent line of development.

It lets you work on a feature without disturbing the main code.

Why it matters

In team projects:

- main stays stable
- feature work happens in separate branches
- after testing, branch is merged back

Commands

git branch

git branch feature-login

git checkout feature-login

Modern way:

git switch -c feature-login

Merge branch

git checkout main
git merge feature-login

Interview line

“Branches let us isolate work so developers can build features independently without breaking the stable codebase.”

Example

You are building:

- branch login-page
- branch payment-page

Both can be developed separately.

9) Merge conflict

What it is

A merge conflict happens when Git cannot decide how to combine changes from two branches or two people edited the same lines.

How to say it

“A merge conflict occurs when the same part of a file is changed in multiple places, and Git needs manual resolution.”

Example

Person A changes line 10 in app.js
Person B also changes line 10 in app.js
Git stops and asks you to resolve it.

How to solve

1. Open the conflicted file
2. Find conflict markers:
3. <<<<<< HEAD
4. your code
5. =====
6. incoming code
7. >>>>>> branch-name

8. Keep the correct code
9. Remove markers
10. Add and commit again

Interview tip

Do not pretend merge conflicts are rare. They happen often in teams.

10) Fetch vs Pull

This is a classic interview question.

git fetch

Downloads remote changes but does not merge them.

git pull

Downloads remote changes and merges them into your current branch.

How to say it

“fetch brings remote updates to my local repository without changing my working branch, while pull fetches plus merges.”

Example

If you want to inspect changes first:

```
git fetch origin  
git log origin/main
```

Then merge later.

11) Clone vs Fork

git clone

Copies a repository to your local machine.

Example

```
git clone https://github.com/user/repo.git
```

Fork

Creates your own copy of someone else's GitHub repo under your GitHub account.

How to say it

"Cloning downloads a repository locally. Forking creates a separate GitHub copy under my account, which is useful for contributing to projects I do not own."

Example

- Clone: when you want to work locally
 - Fork: when you want to contribute to someone else's public repo
-

12) GitHub Pages deployment

If you host a simple static site on GitHub Pages, the usual process is:

1. Push HTML/CSS/JS to GitHub
2. Go to repo settings
3. Enable GitHub Pages
4. Choose branch like main
5. Save and get a public URL

Interview line

"GitHub Pages lets me host static websites directly from a GitHub repository."

Important limitation

GitHub Pages is good for:

- static websites
- portfolios
- documentation

It is not for normal backend apps unless you only host frontend there.

13) File optimization before upload

This matters for real-world projects.

Images

Use:

- .webp instead of heavy .png/.jpg when possible

Fonts

Use:

- .woff2 instead of heavy .ttf

Why

Smaller assets load faster and improve website performance.

Interview line

“I optimize assets before deployment because large images and fonts increase load time and hurt performance.”

Example

Instead of:

- logo.png 2 MB

Use:

- logo.webp 120 KB

That is a real improvement.

14) Important commands you should know

These come up often.

Check changes

git diff

Shows unstaged changes.

See staged changes

git diff --cached

Remove file from staging

git restore --staged filename

Discard local changes

git restore filename

Create and switch branch

```
git switch -c new-branch
```

Switch branch

```
git switch main
```

Rename branch

```
git branch -m new-name
```

Delete branch

```
git branch -d feature-branch
```

Show branch history graph

```
git log --oneline --graph --all
```

15) Hard interview queries you must prepare

These are the ones that expose shallow understanding.

1. What is the difference between working directory, staging area, and repository?

Answer

- Working directory: where you edit files
- Staging area: files prepared for commit
- Repository: saved commit history

How to learn

Do one file change and observe:

```
git status  
git add file  
git status  
git commit -m "msg"
```

2. What is HEAD?

Answer

HEAD points to the current commit or branch you are on.

Simple explanation

It tells Git “where you are right now”.

3. reset vs revert

git reset

Moves branch history backward. Can rewrite history.

git revert

Creates a new commit that undoes a previous commit. Safer for shared branches.

Interview line

“I use reset for local history correction and revert when I want to safely undo changes on shared branches.”

Learn this carefully

This is a common confusion point.

4. merge vs rebase

Merge

Combines histories and keeps branch structure.

Rebase

Reapplies commits on top of another base, making history linear.

Interview line

“Merge preserves branch history, while rebase rewrites commits onto a new base for a cleaner linear history.”

Brutally honest note

Do not claim to understand rebase if you only watched one video. Practice it locally first.

5. What is .gitignore?

Answer

A file that tells Git which files to ignore.

Example

```
node_modules/  
.env  
*.log
```

Why it matters

You should never commit:

- secrets
 - API keys
 - build files
 - dependencies like node_modules
-

6. What is a commit hash?

Answer

A unique identifier for a commit.

Example

a1b2c3d

It helps track a specific version exactly.

16) Extra interview topics you should cover because they are often asked

These were not in your notes, but they matter.

Stash

Temporarily save unfinished changes.

Command

```
git stash  
git stash pop
```

Use case

You are in the middle of work but need to switch branches quickly.

Remote branch tracking

A local branch can track a remote branch.

Example

```
git push -u origin main
```

-u sets upstream tracking.

Pull request

A GitHub feature for reviewing and merging code.

Interview line

“A pull request is a request to merge branch changes into another branch after review.”

Code review

Other developers inspect your code before merge.

Why it matters:

- catches bugs
 - improves code quality
 - enforces standards
-

Git workflow in teams

A common flow:

1. Clone repo
2. Create branch
3. Make changes
4. Commit
5. Push branch
6. Open pull request
7. Review

8. Merge

You should know this sequence cleanly.

17) What to say in interviews: ready-made answers

Git vs GitHub

“Git is a local version control system. GitHub is a remote hosting platform for Git repositories.”

Why Git?

“Git helps track history, collaborate safely, and recover older versions if needed.”

What is a commit?

“A commit is a saved snapshot of my project changes with a message.”

What is a branch?

“A branch is an isolated line of development used to work on features without affecting the stable code.”

What is a merge conflict?

“It happens when Git cannot automatically combine changes from multiple sources.”

Fetch vs pull

“Fetch only downloads changes; pull downloads and merges.”

Clone vs fork

“Clone makes a local copy; fork makes a copy in your GitHub account.”

18) How to learn the hard parts properly

Do not memorize Git like a theory subject. Git only sticks when you break and fix things yourself.

Best learning method

Create a dummy repo and practice this sequence:

```
git init
touch a.txt
git add a.txt
git commit -m "add a.txt"
echo "hello" >> a.txt
git status
git diff
git add a.txt
git commit -m "update a.txt"
git log --oneline
```

Then practice:

- branch creation
- merge
- conflict resolution
- stash
- reset and revert

Order to learn

1. init, add, commit, status, log
2. remote, push, pull
3. branch, merge
4. fetch vs pull
5. stash
6. reset vs revert
7. rebase
8. pull request workflow

That order is much better than jumping to advanced commands first.

19) One simple real-world story you can use in interview

“I worked on a project where multiple features were being added at the same time. I used Git branches so each feature stayed isolated. After testing, I pushed the branch to GitHub and created a pull request. When a merge conflict happened, I manually resolved the conflicting lines, committed the fix, and merged it back safely. That workflow kept the main branch stable.”

That sounds practical and real. Interviewers like that more than definitions.

20) The minimum command set you should memorize

```
git init
git status
```

git add .
git commit -m "message"
git log --oneline
git remote add origin <url>
git push origin main
git pull origin main
git branch
git switch -c feature
git merge feature
git clone <url>
git fetch origin
git stash
git diff
git restore
git revert
git reset

If you want, I can turn this into a **proper Git interview cheat sheet** in a clean Q&A format with **very short answers + one-line examples + common follow-up questions**.